705.641.81: Natural Language Processing Self-Supervised Models

Homework 5: RNNs and Transformers/Self-Attention

For homework deadline and collaboration policy, please see our Canvas page.

Name: Jose Luis Montalvo

Collaborators, if any: _____

Sources used for your homework, if any: _____

This assignment focuses on basic understanding of Recurrent Neural Networks and Transformers for language modeling. We will also touch upon Tokenization.

Homework goals: After completing this homework, you should be comfortable with:

- aligning your prior understanding of Neural Net training for RNNs;
- thinking about Self-Attention module and implementing a simple Transformer language model.

Concepts, intuitions and big picture

Each question might have multiple correct answers. (Check all that apply).

1. Which of these types of models would you use for completing prompts with generated text?
   - ☐ An encoder model
   - ☑ A decoder model

2. Which of these types of models would you use for classifying text inputs according to certain labels?
   - ☑ An encoder model
   - ☐ A decoder model

3. To which of these tasks would you apply a many-to-one RNN architecture?
   - ☐ Speech recognition (input an audio clip and output a transcript)
   - ☑ Sentiment classification (input a piece of text and output a 0/1 to denote positive or negative sentiment)
   - ☑ Gender recognition from speech (input an audio clip and output a label indicating the speaker's gender)

4. What possible source can the bias observed in a model have?
   - ☑ The model is a fine-tuned version of a pretrained model and it picked up its bias from it.
   - ☑ The data the model was trained on is biased.
   - ☑ The metric the model was optimizing for is biased.

5. How many dimensions does the tensor output by a decoder Transformer model have, and what are they?
   - ☐ 2: The sequence length and the batch size
   - ☐ 2: The sequence length and the hidden size
   - ☑ 3: The sequence length, the batch size, and the hidden size

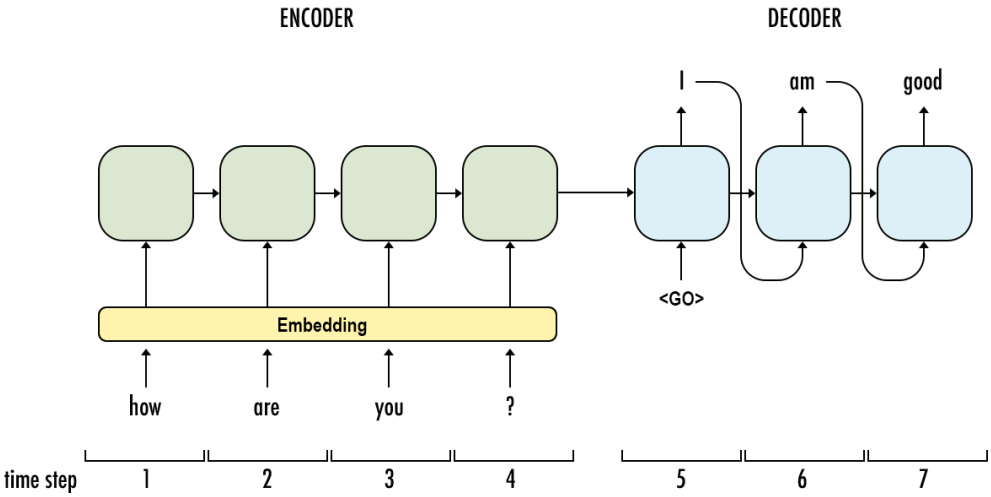
6. You are training an RNN language model. At the tth time step, what is the RNN doing? Choose the best answer.
   - ☐ Estimating P$(y_1, y_2, \ldots, y_{t-1})$
   - ☐ Estimating P(y_1)
   - ☑ Estimating P$(y_t | y_1, y_2, \ldots, y_{t-1})$
   - ☐ Estimating P$(y_t | y_1, y_2, \ldots, y_t)$

7. You are training an RNN, and find that your weights and activations are all taking on the value of `NaN` ("Not a Number"). Which of these is the most likely cause of this problem?
   - [ ] Vanishing gradient problem.
   - [x] Exploding gradient problem.
   - [ ] ReLU activation function g(.) used to compute g(z), where z is too large.
   - [ ] Sigmoid activation function g(.) used to compute g(z), where z is too large.

8. You're done training an RNN language model. You're usig it to sample random sentences as follows. What are you doing at each time step t?
   - [ ] (i) Use the probabilities output by the RNN to pick the highest probability word for that time-step as y_t. (ii) Then pass the ground-truth word from the training set to the next time-step.
   - [ ] (i) Use the probabilities output by the RNN to randomly sample a chosen word for that time-step as y_t. (ii) Then pass the ground-truth word from the training set to the next time-step.
   - [ ] (i) Use the probabilities output by the RNN to pick the highest probability word for that time-step as y_t. (ii) Then pass this selected word to the next time-step.
   - [x] (i) Use the probabilities output by the RNN to randomly sample a chosen word for that time-step as y_t. (ii) Then pass this selected word to the next time-step.

9. You have a pet crab whose mood is heavily dependent on the current and past few days' weather. You've collected data for the past 42 days on the weather, which you represent as a sequence as $x_1, \ldots, x_{42}$. You've also collected data on your crab's mood, which you represent as $y_1, \ldots, y_{42}$. You'd like to build a model to map from $x \rightarrow y$. Should you use a Unidirectional RNN or Bidirectional RNN for this problem?
   - [ ] Bidirectional RNN, because this allows the prediction of mood on day t to consider more information.
   - [ ] Bidirectional RNN, because this allows backpropagation to compute more accurate gradients.
   - [x] Unidirectional RNN, because the value of y_t depends only on $x_1, \ldots, x_t$ but not on $x_{t+1}, \ldots, x_{42}$
   - [ ] Unidirectional RNN, because the value of y_t depends only on x, and not other days' weather.

10. Consider this encoder-decoder model.



This model is a "conditional language model" in the sense that the encoder portion (shown in green) is modeling the probability of the input sentence x.

☐ True ☑ False

11. Compared to the RNN-LMs and fixed-window-LMs, we expect the attention-based LMs to have the greatest advantage when:

☑ The input sequence length is large.

☐ The input sequence length is small.

12. What is a model head?

☐ A component of the base Transformer network that redirects tensors to their correct layers

☐ Also known as the self-attention mechanism, it adapts the representation of a token according to the other tokens of the sequence

☑ An additional component, usually made up of one or a few layers, to convert the transformer predictions to a task-specific output

13. What are the techniques to be aware of when batching sequences of different lengths together?

☑ Truncating ☐ Returning tensors ☑ Padding ☑ Attention masking

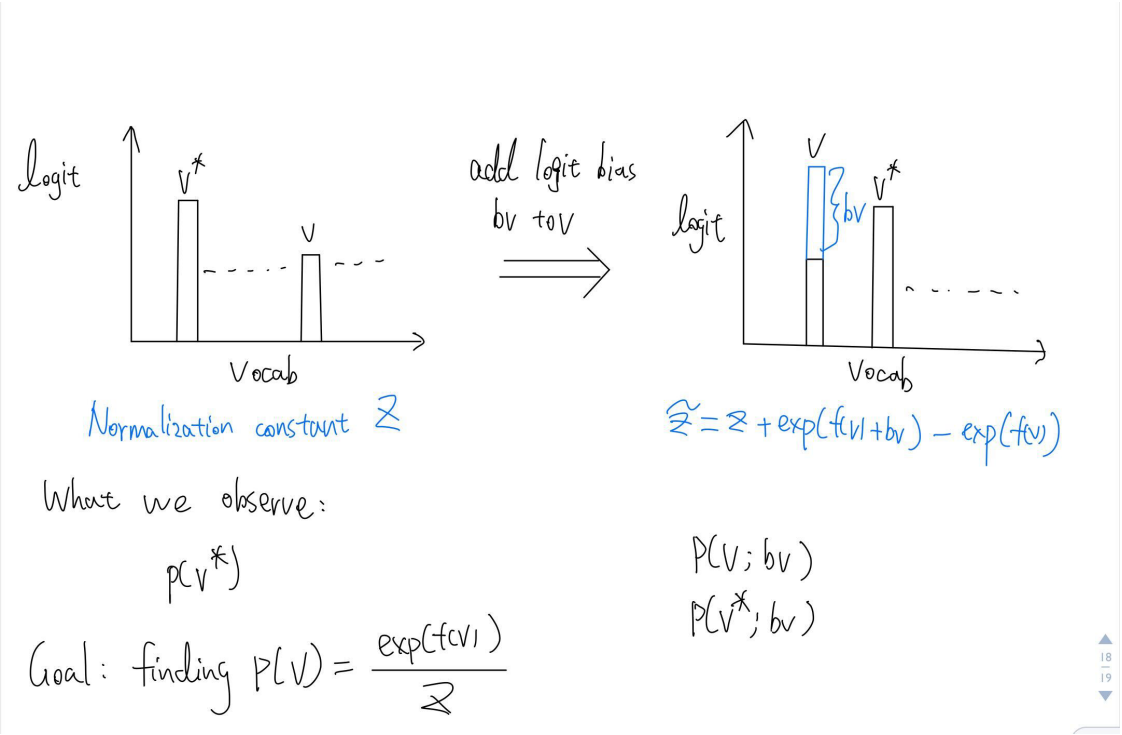
Extra Credit: Inferring the whole distribution from a semi-closed API

It's that time of the year! OpenAI frequently releases new language models that beat the hell out of all the language models out there. Jiefu, your boss at the startup is asking you to do cool stuff with this LM such as top-p sampling, but you realize that this API does not output the full probability distribution of the words, it only outputs the probabilities of the top-2 words. But you know that Jiefu does not take "no" for answer! So you need to figure out a way to identify the full distribution of probabilities over words.

All of a sudden, you realize that OpenAI has implemented a feature called "logit bias" that returns the distribution of the top-2 words after adding a bias value to a particular word in the **logit** distribution. This bias term is to the logits generated by the model prior to the normalization of the logits into a probability and returning the top-2 values. Logit bias is useful for up-weighting/down-weighting word probabilities in downstream applications that want a certain word to never/always appear in the output.

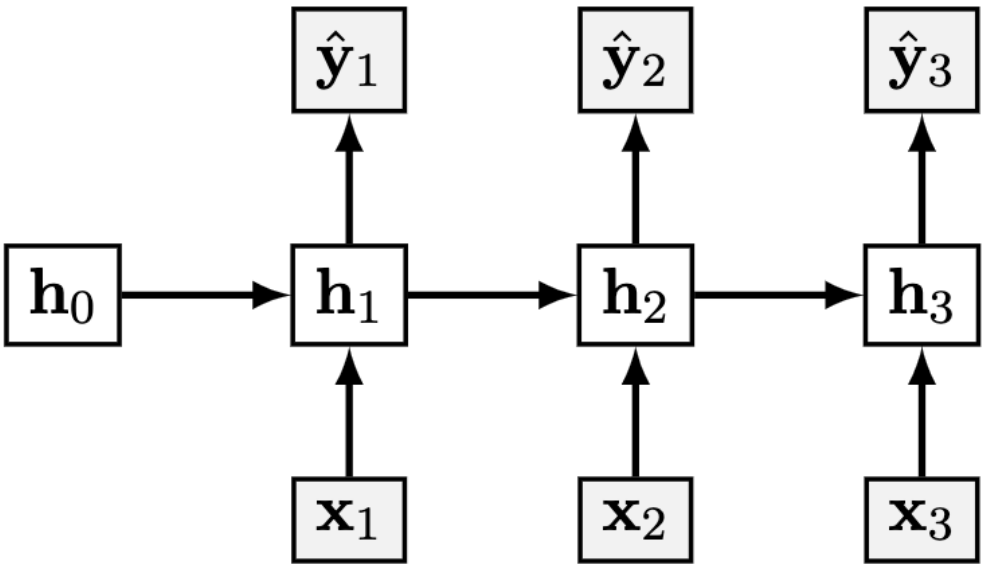
Your task is to find a way to logit bias identify the probability of all the words in the distribution. You algorithm needs accomplish this goal by at most $|V| + 1$ API calls (for a vocabulary of size $|V|$).

Hints: The figure below provides some hints into how to think about this problem. Initially (left) you observe v^* as the most likely word. You do not really observe the probability of v because the API does not expose it but you know that it's there in the distribution. Then since you want to identify the probability of v, you add b_v logit bias to this word to make it the highest probability word and you get a modified distribution (right). Notice that, these two distributions have different normalizing constants. If the left distribution has a normalizing constant of Z, the normalizing constant of the distribution on the right size would be $\sim Z = Z + \exp(f(v) + b_v) - \exp\big(f(v)\big)$, where $f(v)$ is the original logit value of v. Also, notice that you observe the probability of v^* as $p(v^*)$ initially, and the modified probability of both v^* and v as $p(v^*; b_v)$ and $p(v; b_v)$ after logit bias added. Given these connections between the two distributions, you can infer $p(v) = \frac{f(v)}{Z}$.



Training a Recurrent Neural Net

Consider the following simple RNN architecture:



Where the layers and their corresponding weights are given below:

$$\begin{aligned} \mathbf{x}_t &\in \mathbb{R}^3 & \mathbf{W}_{hx} &\in \mathbb{R}^{4\times 3} \\ \mathbf{h}_t &\in \mathbb{R}^4 & \mathbf{W}_{yh} &\in \mathbb{R}^{2\times 4} \\ \mathbf{y}_t, \hat{\mathbf{y}}_t &\in \mathbb{R}^2 & \mathbf{W}_{hh} &\in \mathbb{R}^{4\times 4} \end{aligned}$$

$$\begin{aligned} J &= -\sum_{t=1}^{3}\sum_{i=1}^{2} y_{t,i}\log\left(\hat{y}_{t,i}\right) \\ \hat{\mathbf{y}}_t &= \sigma(\mathbf{o}_t) \\ \mathbf{o}_t &= \mathbf{W}_{yh}\mathbf{h}_t \\ \mathbf{h}_t &= \psi(\mathbf{z}_t) \\ \mathbf{z}_t &= \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t \end{aligned}$$

Where σ is the softmax activation and ψ is the identity activation (i.e. no activation).

Forward pass (for t=1,2,3).

$$\mathbf{z_t} = \mathbf{W_{hhh_{t-1}}} + \mathbf{W_{hxx_t}},$$

$$\mathbf{h_t} = \psi(\mathbf{z_t}) = \mathbf{z_t},$$

$$\mathbf{o_t} = \mathbf{W_{yhh_t}},$$

$$\widehat{\mathbf{y}_t} = \sigma(\mathbf{o_t}) = \mathrm{softmax}(\mathbf{o_t}).$$

Loss (sum of per-step cross-entropy).

$$J = -\sum_{t=1}^{3}\sum_{i=1}^{2} y_{t,i}\ \log!\left(\widehat{y_{t,i}}\right).$$

Output delta (softmax + cross-entropy).

$$[\boldsymbol{\delta}_{\boldsymbol{t}}^{(o)} \equiv \frac{\partial J}{\partial \mathbf{o_t}} = \widehat{\mathbf{y}_t} - \mathbf{y_t} \in R^{\mathbb{2}}. = \widehat{\mathbf{y}_t} - \mathbf{y_t} \in R^{\mathbb{2}}]$$

Backpropagation through time (BPTT).

$$\boldsymbol{\delta}_{\boldsymbol{t}}^{(\boldsymbol{z})} \equiv \frac{\partial J}{\partial \mathbf{z_t}} \in R^{\mathbb{4}} and set \boldsymbol{\delta}_{\mathbf{4}}^{(\boldsymbol{z})} = \mathbf{0}. Then,$$

$$[\boldsymbol{\delta}_{\boldsymbol{t}}^{(\boldsymbol{z})} = \mathbf{W}_{\mathbf{yh}}^{\top \boldsymbol{\delta}_{\mathbf{t}}^{(\mathbf{o})}} + \mathbf{W}_{\mathbf{hh}}^{\top \boldsymbol{\delta}_{\mathbf{t+1}}^{(\mathbf{z})}}.$$

Parameter gradients.

$$\backslash begin\{align *\}$$

$$\frac{\partial J}{\partial \mathbf{W_{yh}}} = \sum_{t=1}^{3} \boldsymbol{\delta}_t^{(o)} \ \mathbf{h_t}^\top$$

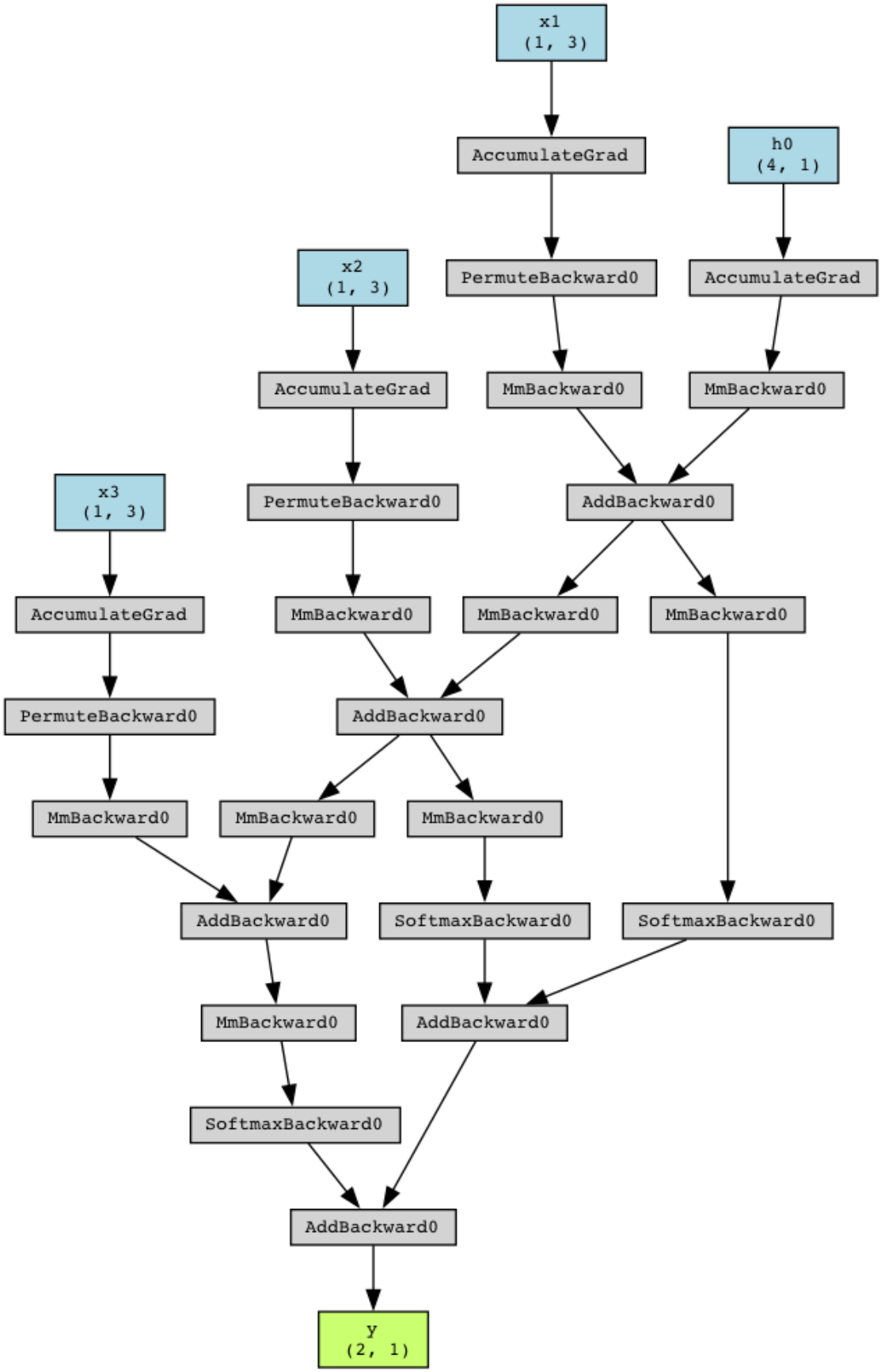
$$= \sum_{t=1}^{3} (\widehat{\mathbf{y}}_t - \mathbf{y_t}) \ \mathbf{h_t}^\top \in R^{2 \times 4}, \backslash$$

$$\frac{\partial J}{\partial \mathbf{W_{hh}}} = \sum_{t=1}^{3} \boldsymbol{\delta}_t^{(z)} \ \mathbf{h_{t-1}}^\top \in R^{4 \times 4}, \backslash$$

$$\frac{\partial J}{\partial \mathbf{W_{hx}}} = \sum_{t=1}^{3} \boldsymbol{\delta}_t^{(z)} \ \mathbf{x_t}^\top \in R^{4 \times 3}.$$

Computation Graph

Draw the computational graph for the given model.


x1
(1, 3)
AccumulateGrad
h0
(4, 1)
x2
(1, 3)
PermuteBackward0
AccumulateGrad
AccumulateGrad
MmBackward0
MmBackward0
x3
(1, 3)
PermuteBackward0
AddBackward0
AccumulateGrad
MmBackward0
MmBackward0
MmBackward0
PermuteBackward0
AddBackward0
MmBackward0
MmBackward0
MmBackward0
AddBackward0
SoftmaxBackward0
SoftmaxBackward0
MmBackward0
AddBackward0
SoftmaxBackward0
AddBackward0
y
(2, 1)

Q2.1 Computation graph.

Backprop for RNN

Now you will derive the steps of the backpropagation algorithm that lead to the computation of $\frac{dJ}{dW_{hh}}$. For all parts of this question, please write your answer in terms of $W_{hh}, W_{yh}, y, \hat{y}, h$, and any additional terms specified in the question (note: this does not mean that every term listed shows up in every answer, but rather that you should simplify terms into these as much as possible when you can).

1. Let's define a cross-entropy for our RNN at time t: $J_t = -\sum_{i=1}^{2} y_{t,i} \log(\hat{y}_{t,i})$. What is $\frac{\partial J_t}{\partial o_t}$? Write your answer and show your work. **Hint:** First derive the partial derivatives with respective to everything after h_t. After that, try to break up the objective into a term for each timestep.

$$\frac{\partial J_t}{\partial \widehat{y_{t,\iota}}} = -\frac{y_{t,i}}{\widehat{y_{t,\iota}}},$$

$$\frac{\partial \widehat{y_{t,\iota}}}{\partial o_{t,j}} = \widehat{y_{t,\iota}}(\delta_{ij} - \widehat{y_{t,j}}).$$

Hence

$$\frac{\partial J_t}{\partial o_{t,j}} = \sum_i \frac{\partial J_t}{\partial \widehat{y_{t,\iota}}} \frac{\partial \widehat{y_{t,\iota}}}{\partial o_{t,j}} = \sum_i !\left(-\frac{y_{t,i}}{\widehat{y_{t,\iota}}}\right) \widehat{y_{t,\iota}}(\delta_{ij} - \widehat{y_{t,j}}) = \widehat{y_{t,j}} - y_{t,j}.$$

2. Suppose you have a variable $\mathbf{g}_{\mathbf{o}_t}$ that stores the value of $\frac{\partial J_t}{\partial \mathbf{o}_t}$. What is $\frac{\partial J_t}{\partial \mathbf{h}_i}$ for an arbitrary $i \in [1,3]$ ($i \leq t$)? Write your solution in terms of the $\mathbf{g}_{\mathbf{o}_t}$ and the aforementioned variables and show your work.

$$g_{h_t} \coloneqq \frac{dJ_t}{dh_t} = W_{yh}^T \, g_{o_t}$$

Propagating through the recurrent links,

$$\frac{dJ_t}{dh_{t-1}} = W_{hh}^T \frac{dJ_t}{dh_t}$$

$$\frac{dJ_t}{dh_i} = (W_{hh}^T)^{t-i} \frac{dJ_t}{dh_t}$$

Therefore,

$$\frac{dJ_t}{dh_i} = (W_{hh}^T)^{t-i} W_{yh}^T g_{o_t}$$

$$\frac{dJ_t}{dh_i} = (W_{hh}^T)^{t-i} W_{yh}^T (\widehat{y_t} - y_t)$$

3. Suppose you have a variable $\mathbf{g}_{\mathbf{h}_i}$ that stores the value of $\frac{\partial J_t}{\partial \mathbf{h}_i}$. What is $\frac{\partial J_t}{\partial \mathbf{W}_{hh}}$? Write your solution in terms of the $\mathbf{g}_{\mathbf{h}_i}$ and the aforementioned variables. Show your work in the second.

$$dJ_t/dW_{hh}$$

(given)

$$g_{h_i} = dJ_t/dh_i$$

Because

$$h_k = W_{hh}h_{k-1} + \cdots$$

$$\frac{dh_k}{dW_{hh}} = h_{k-1}^T$$

Each step k <= t contributes

$$g_{h_k} h_{k-1}^T$$

Summing:

$$\frac{dJ_t}{dW_{hh}} = \sum_{k=1}^{t} g_{h_k} h_{k-1}^T$$

$$\frac{dJ_t}{dW_{hh}} = \sum_{k=1}^{t} \left[(W_{hh}^T)^{t-k} W_{yh}^T g_{o_t}\right] h_{k-1}^T$$

$$\frac{dJ_t}{dW_{hh}} = \sum_{k=1}^{t} \left[(W_{hh}^T)^{t-k} W_{yh}^T (\widehat{y_t} - y_t)\right] h_{k-1}^T$$

4. Suppose you have a variable $\mathbf{g}_{W_{hh},t}$ that stores the value of $\frac{\partial J_t}{\partial W_{hh}}$. What is $\frac{\partial J}{\partial \mathbf{w}_{hh}}$? Write your solution in terms of the $\mathbf{g}_{W_{hh},t}$ and the aforementioned variables. Show your work.

r5.5cm

$$dJ/dW_{hh} \quad given \quad g_{W_{hh},t} = dJ_t/dW_{hh}$$

With

$$J = \sum_{t=1}^{3} J_t$$

$$\frac{dJ}{dW_{hh}} = \sum_{t=1}^{3} g_{W_{hh},t}$$

$$\frac{dJ}{dW_{hh}} = \sum_{t=1}^{3}\sum_{k=1}^{t}\left[(W_{hh}^T)^{t-k} W_{yh}^T g_{o_t}\right] h_{k-1}^T$$

$$\frac{dJ}{dW_{hh}} = \sum_{t=1}^{3}\sum_{k=1}^{t}\left[(W_{hh}^T)^{t-k} W_{yh}^T (\hat{y}_t - y_t)\right] h_{k-1}^T$$

Self-Attention and Transformers

Recall that the transformer architecture uses scaled dot-product attention to compute *attention weights*:

$$\boldsymbol{\alpha}^{(t)} = \text{softmax}\left(\frac{\mathrm{q}_t \mathrm{K}^\top}{\sqrt{h}}\right) \in [0,1]^n$$

The resulting embedding in the output of attention at position t are:

$$\text{Attention(Q,K,V)}_t = \sum_{t\prime=1}^{n} \alpha_{t\prime}^{(t)} \mathrm{v}_{t\prime} \in \mathbb{R}^{1\times h},$$

where $\boldsymbol{\alpha}^{(t)} = \left[\alpha_0^{(t)}, \ldots, \alpha_n^{(t)}\right]$. The same idea can be stated in a matrix form,

$$\text{Attention(Q,K,V)} = \text{softmax}\left(\frac{\mathrm{QK}^\top}{\sqrt{h}}\right)\mathrm{V} \in \mathbb{R}^{n\times h}.$$

In the above equations:

- h is the hidden state dimension and n is the input sequence length;
- $X \in \mathbb{R}^{n \times h}$ is the input to the attention;
- $x_t \in \mathbb{R}^{1 \times h}$ is the slice of X at position t, i.e. vector representation (embedding) of the input token at position t;
- $W_q, W_k, W_v \in \mathbb{R}^{h \times h}$ are the projection matrices to build query, key and value representations;
- $Q = XW_q \in \mathbb{R}^{n \times h}$, $K = XW_k \in \mathbb{R}^{n \times h}$, $V = XW_v \in \mathbb{R}^{n \times h}$ are the query, key and value representations;
- $q_t = x_t W_q \in \mathbb{R}^{1 \times h}$ is the slice of Q at position t. Similarly, $k_t = x_t W_k \in \mathbb{R}^{1 \times h}$ and $v_t = x_t W_v \in \mathbb{R}^{1 \times h}$.

Now answer the following questions:

Complexity

What is the computational complexity of self-attention layer in terms of n and h? In particular, show that the complexity of a self-attention layer at test-time scales quadratically with the sequence length n. Lastly, explain (no more than 5 sentences) why this can be computed efficiently on GPUs, despite being a quadratic function of sequence length n.

Total self-attention cost:

$$\boxed{O(nh^2) + O(n^2h)}$$

For fixed hidden size h, the $O(n^2h)$terms dominate, so test-time cost scales quadratically in n(and memory is also $O(n^2)$for the attention matrix).

Why still fast on GPUs: The quadratic pieces are just large, dense matrix–matrix multiplies (GEMMs). GPUs excel at GEMMs via massive data-parallelism, high memory bandwidth, and fused kernels.These operations map to contiguous memory and batched layouts, keeping the compute units saturated. Thus, despite n^2scaling, the work is highly vectorizable and efficiently pipelineable.

Masking in Self-Attention

Suppose we are are using token-making objective to create a language generation model that uses self-attention. For example, suppose we want to mask $t = 3$ position. Then, it does not make sense for $q_t: \forall t \in [0 \dots n]$ to look at k_3 and v_3. Describe one way we can go about implementing such masking.

Implement a mask matrix $M \in \mathbb{R}^{n \times n}$ added to the attention logits before softmax:

$$\text{Attention}(Q, K, V) = \text{softmax}\,,(\frac{QK^\top}{\sqrt{h}} + M)V.$$

To forbid all queries from attending to position $t = 3$, set the 3rd column of M to $-\infty$ (or a very large negative number): $M_{i,3} = -\infty$ for all i, and $M_{i,j} = 0$ otherwise. This zeroes $\alpha_3^{(i)}$ after softmax, so no output uses k_3 or v_3.

Extra Credit: Linear Attention with SVD

It has been empirically shown in Transformer models that the context mapping matrix $P = \text{softmax}\left(\frac{QK^\top}{\sqrt{h}}\right)$ often has a low rank. Show that if the rank of P is k and we already have access to the SVD of P, then it is possible to compute self-attention in $O(nkh)$ time.

Extra Credit: Linear Attention by Projection

Suppose we ignore the Softmax and scaling and let $P = \frac{QK^\top}{\sqrt{h}} \in \mathbb{R}^{n \times n}$. Assume P is rank k. Show that there exist two linear projection matrices $C, D \in \mathbb{R}^{k \times n}$ such that $PV = Q(CK)^\top DV$ and the right hand side can be computed in $O(nkh)$ time. **Hint:** Consider using SVD in your proof.

Programming

In this programming homework, we will

- implement our own GPT - Transformer-based language model.
- learn about how to run Slurm jobs to train and evaluate the GPT LM on GPUs.

Skeleton Code and Structure:

The code base for this homework can be found at this GitHub repo under the `hw5` directory. Your task is to fill in the missing parts in the skeleton code, following the requirements, guidance, and tips provided in this pdf and the comments in the corresponding .py files. The code base has the following structure:

- `gpt` defines the Python module of the GPT LM
    - `bpe.py` creates a Byte-Pair Encoding tokenizer from scratch.
    - `model.py` implements our GPT model architecture from scratch.
    - `trainer.py` is the script for training, evaluating the GPT model, and visualizing the results.
    - `utils.py` defines helper functions.
- `main.py` provides the entry point to run your implementations of `gpt` module.
- `sbatch.sh` provides the bash script for submitting jobs to Slurm (details in 5.1.9)
- `hw5.md` provides instructions on how to setup the environment and run each part of the homework in `main.py`.
- `generate.py` provides helper functions for sampling from GPT LMs.
- `data.py` convert WikiText Data into LM training data.

TODOs — Your tasks include 1) generate plots and/or write short answers based on the results of running the code; 2) fill in the blanks in the skeleton to complete the code. We will explicitly mark these plotting, written answer, and filling-in-the-blank tasks as **TODOs** in the following descriptions, as well as a **`# TODO`** at the corresponding blank in the code.

Submission:

Your submission should contain two parts: 1) plots and short answers under the corresponding questions below; and 2) your completion of the skeleton code base, in a `.zip` file

A GPT Language Model from Scratch

Introduction: Self-attention and Transformer

What do BERT, RoBERTa, GPT4, … all have in common? *Self-attention and Transformer-based architecture*! Transformer-based architectures, which are primarily used in modeling language understanding tasks, eschew the use of recurrence in neural networks

(RNNs) and instead trust entirely on self-attention mechanisms to draw global dependencies between inputs and outputs.

Some useful documentation:

As the architecture is so popular, there already exists a Pytorch module for `nn.Transformer` and a tutorial on how to use it for the next token prediction. However, we will implement it here ourselves, to get through to the smallest details.

There are of course many more tutorials out there about attention and Transformers. Below, we list a few that are worth exploring if you are interested in the topic and might want yet another perspective on the topic after this one:

- Transformer: A Novel Neural Network Architecture for Language Understanding (Jakob Uszkoreit, 2017) – The original Google blog post about the Transformer paper, focusing on the application in machine translation.
- The Illustrated Transformer (Jay Alammar, 2018) – A very popular and great blog post intuitively explaining the Transformer architecture with many nice visualizations.
- Attention? Attention! (Lilian Weng, 2018) – A nice blog post summarizing attention mechanisms in many domains including vision.
- Illustrated: Self-Attention (Raimi Karim, 2019) – A nice visualization of the steps of self-attention. Recommended going through if the explanation below is too abstract for you.
- The Transformer family (Lilian Weng, 2020) – A very detailed blog post reviewing more variants of Transformers besides the original one.

What is Attention?

The attention mechanism describes a recent new group of layers in neural networks that has attracted a lot of interest in the past few years, especially in sequence tasks. There are a lot of different possible definitions of “attention" in the literature, but the one we will use here is the following: *the attention mechanism describes a weighted average of (sequence) elements with the weights dynamically computed based on an input query and elements’ keys*. So what does this exactly mean? The goal is to take an average of the features of multiple elements. However, instead of weighting each element equally, we want to weight them depending on their actual values. In other words, we want to dynamically decide on which inputs we want to “attend" more than others. In particular, an attention mechanism has usually four parts we need to specify:

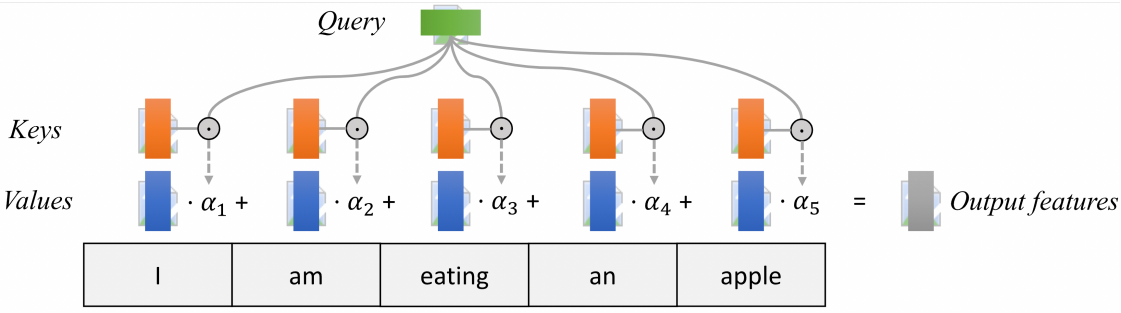
- **Query**: The query is a feature vector that describes what we are looking for in the sequence, i.e. what would we maybe want to pay attention to.

- **Keys**: For each input element, we have a key which is again a feature vector. This feature vector roughly describes what the element is "offering", or when it might be important. The keys should be designed such that we can identify the elements we want to pay attention to based on the query.
- **Values**: For each input element, we also have a value vector. This feature vector is the one we want to average over.
- **Score function**: To rate which elements we want to pay attention to, we need to specify a score function f_{attn}. The score function takes the query and a key as input, and outputs the score/attention weight of the query-key pair. It is usually implemented by simple similarity metrics like a dot product.

The weights of the average are calculated by a Softmax overall score function outputs. Hence, we assign those value vectors a higher weight whose corresponding key is most similar to the query. If we try to describe it with pseudo-math, we can write:

$$\alpha_i = \frac{\exp\left(f_{attn}(\text{key}_i, \text{query})\right)}{\sum_j \exp\left(f_{attn}\left(\text{key}_j, \text{query}\right)\right)}, \qquad \text{out} = \sum_i \alpha_i \cdot \text{value}_i$$

Visually, we can show the attention over a sequence of words as follows. For every word, we have one key and one value vector. The query is compared to all keys with a score function (in this case the dot product) to determine the weights. The Softmax is not visualized for simplicity. Finally, the value vectors of all words are averaged using the attention weights.



Most attention mechanisms differ in terms of what queries they use, how the key and value vectors are defined, and what score function is used. The attention applied inside the Transformer architecture is called **self-attention**. In self-attention, each sequence element provides a key, value, and query. For each element, we perform an attention layer where based on its query, we check the similarity of all sequence elements' keys and returned a different, averaged value vector for each element. We will now go into a bit more detail by first looking at the specific implementation of the attention mechanism which is in the Transformer case the scaled dot product attention.

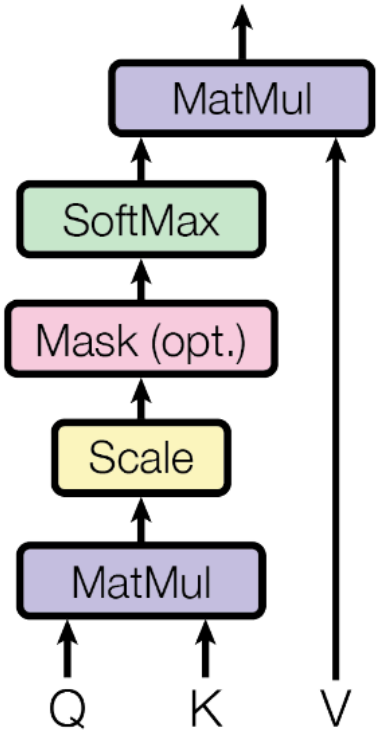
Scaled Dot Product Attention

The core concept behind self-attention is the scaled dot product attention. Our goal is to have an attention mechanism with which any element in a sequence can attend to any other while still being efficient to compute. The dot product attention takes as input a set of queries $Q \in \mathbb{R}^{T \times d_k}$, keys $K \in \mathbb{R}^{T \times d_k}$ and values $V \in \mathbb{R}^{T \times d_v}$ where T is the sequence length, and d_k and d_v are the hidden dimension for queries/keys and values respectively. For simplicity, we neglect the batch dimension for now. The attention value from element i to j is based on its similarity of the query Q_i and key K_j, using the dot product as the similarity metric. In math, we calculate the dot product attention as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The matrix multiplication QK^T performs the dot product for every possible pair of queries and keys, resulting in a matrix of the shape $T \times T$. Each row represents the attention logits for a specific element i to all other elements in the sequence. On these, we apply a Softmax and multiply with the value vector to obtain a weighted mean (the weights being determined by the attention). Another perspective on this attention mechanism offers the computation graph which is visualized below (figure credit:). There is also a nice visualization of these values are computed here.

Scaled Dot-Product Attention



One aspect we haven't discussed yet is the scaling factor of $1/\sqrt{d_k}$. This scaling factor is crucial to maintain an appropriate variance of attention values after initialization. Remember that we initialize our layers to have equal variance throughout the model, and hence, Q and K might also have a variance close to 1. However, performing a dot product over two vectors with a variance σ results in a scalar having d_k-times higher variance:

$$q_i \sim \mathcal{N}(0,\sigma), k_i \sim \mathcal{N}(0,\sigma) \rightarrow \text{Var}\left(\sum_{i=1}^{d_k} q_i \cdot k_i\right) = \sigma \cdot d_k$$

If we do not scale down the variance back to σ, the softmax over the logits will already saturate to 1 for one random element and 0 for all others. The gradients through the softmax will be close to zero so we can't learn the parameters appropriately.

Multi-Head Attention

The scaled dot product attention allows a network to attend over a sequence. However, often there are multiple different aspects a sequence element wants to attend to, and a single weighted average is not a good option for it. This is why we extend the attention mechanisms to multiple heads, i.e. multiple different query-key-value triplets on the same features. Specifically, given a query, key, and value matrix, we transform those into h sub-

queries, sub-keys, and sub-values, which we pass through the scaled dot product attention independently. Afterward, we concatenate the heads and combine them with a final weight matrix. Mathematically, we can express this operation as:

$$\begin{aligned}\text{Multihead}(Q,K,V) &= \text{Concat}(\text{head}_1,\ldots,\text{head}_h)W^O\\ \text{where head}_i &= \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right)\end{aligned}$$

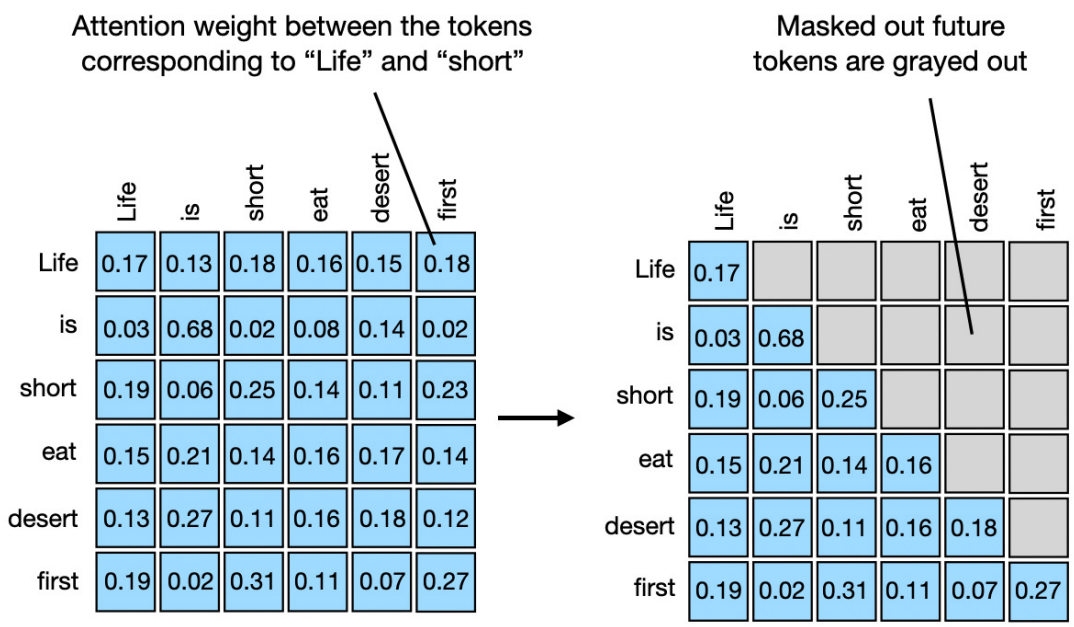
We refer to this as Multi-Head Attention layer with the learnable parameters $W_{1\ldots h}^Q \in \mathbb{R}^{D\times d_k}$, $W_{1\ldots h}^K \in \mathbb{R}^{D\times d_k}$, $W_{1\ldots h}^V \in \mathbb{R}^{D\times d_v}$, and $W^O \in \mathbb{R}^{h\cdot d_k\times d_{out}}$ (D being the input dimension). Expressed in a computational graph, we can visualize it below (figure credit:).

Causal Attention Masking

For GPT-like LMs, we train the model to generate one token at a time from left to right (autoregressive). When predicting the next token, the autoregressive setup constraints the model to only "see", i.e. attend to, the preceding tokens on the left-hand side. For instance, given a sample training text: "Life is short eat dessert first", the self-attention should be applied only within the tokens on the left-hand side of the →, in order to predict the token on the right-hand side:

- "Life" → "is"
- "Life is" → "short"
- "Life is short" → "eat"
- "Life is short eat" → "dessert"
- "Life is short eat dessert" → "first"

To achieve this in practice, one straightforward approach is to mask out future tokens for each timestamp by applying a mask to the attention weight matrix (causal attention masking). For the example above, we can apply the mask:



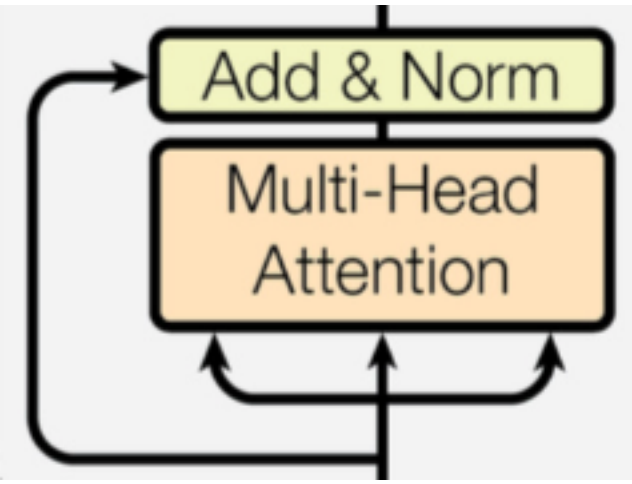
TODOs: Read and complete the missing lines in the `__init__` function of the `CausalSelfAttention` class in `gpt/model.py`, create the causal attention mask.
Hint: Follow the requirements and hints specified in the code comments.

Putting it Together (so far): Multi-Head Self Attention with Causal Masking

Now that we have delved into the details of all important components, it’s time to build the causal self-attention block for our own GPT!
TODOs: Read and complete the missing lines in the `forward` function of the `CausalSelfAttention` class in `gpt/model.py`.
Hint: Follow the requirements and hints specified in the code comments.

Residual connection and normalization

Notice that there is a residual connection and normalization on top of the multi-head attention layer.



Taking as input x, it is first passed through a Multi-Head Attention block as we have implemented above. The output is added to the original input using a residual connection, and we apply a consecutive Layer Normalization on the sum. Overall, it calculates $\text{LayerNorm}\big(x + \text{Multihead}(x, x, x)\big)$ (x being Q, K and V input to the attention layer). The residual connection is crucial in the Transformer architecture for two reasons:

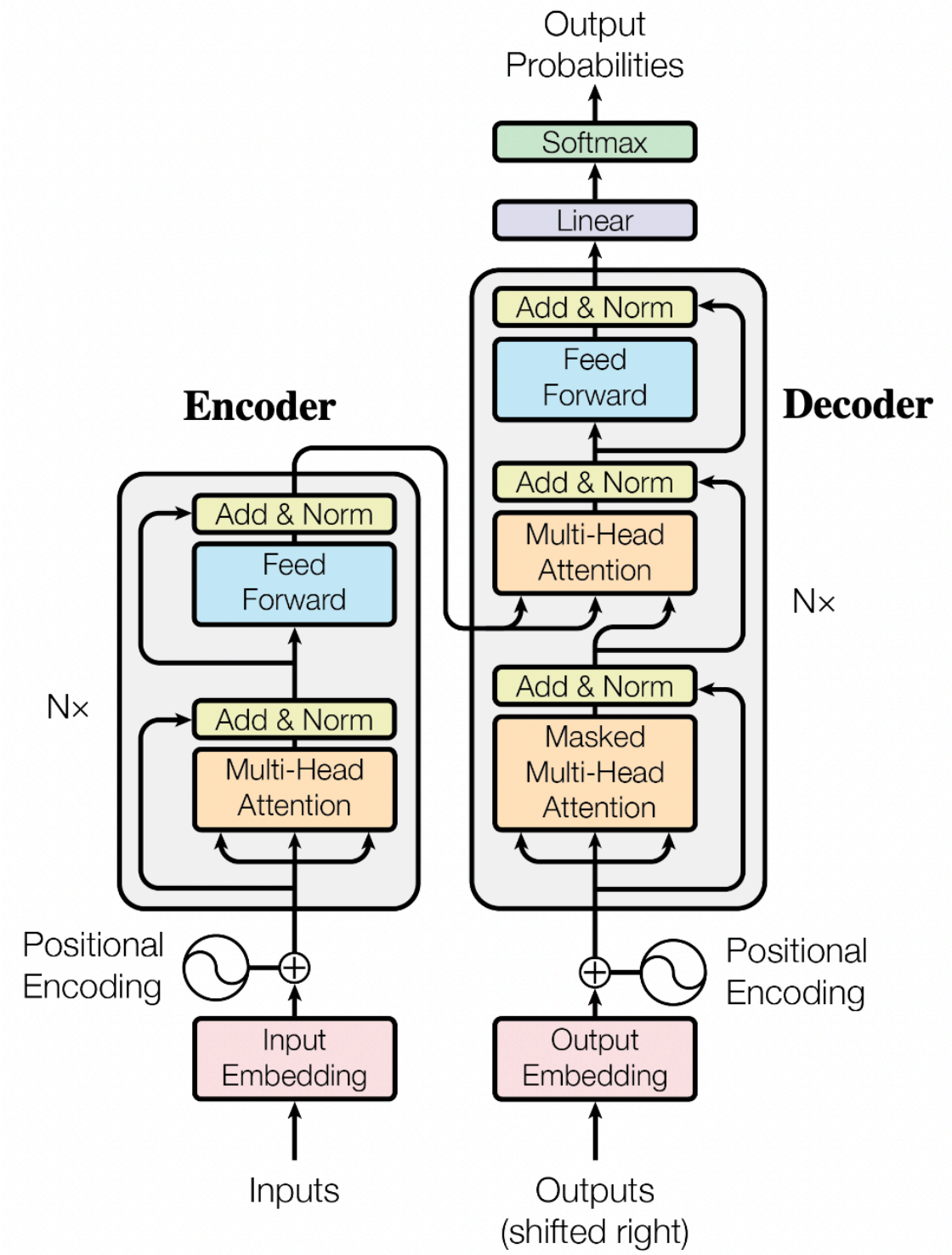
1. Similar to ResNets, Transformers are designed to be very deep. Some models contain more than 24 blocks in the encoder. Hence, the residual connections are crucial for enabling a smooth gradient flow through the model.

2. Without the residual connection, the information about the original sequence is lost. Remember that the Multi-Head Attention layer ignores the position of elements in a sequence, and can only learn it based on the input features. Removing the residual connections would mean that this information is lost after the first attention layer (after initialization), and with a randomly initialized query and key vector, the output vectors for position i have no relation to its original input. All outputs of the attention are likely to represent similar/same information, and there is no chance for the model to distinguish which information came from which input element. An alternative option to residual connection would be to fix at least one head to focus on its original input, but this is very inefficient and does not have the benefit of the improved gradient flow.

The Layer Normalization also plays an important role in the Transformer architecture as it enables faster training and provides small regularization. Additionally, it ensures that the features are in a similar magnitude among the elements in the sequence. We are not using Batch Normalization because it depends on the batch size which is often small with Transformers (they require a lot of GPU memory), and BatchNorm has shown to perform particularly badly in language as the features of words tend to have a much higher variance (there are many, very rare words which need to be considered for a good distribution estimate).

Putting it Together: Transformer Architecture

Originally, the Transformer model was designed for machine translation. Hence, it has an encoder-decoder structure where the encoder takes as input the sentence in the original language and generates an attention-based representation. *This is different from our GPT implementation, which is a decoder-only architecture*. On the other hand, the decoder attends to the encoded information and generates the translated sentence in an auto-regressive manner, as in a standard RNN. The full Transformer architecture looks as follows (figure credit:):



The encoder consists of N identical blocks. These blocks contain Self-Attention, Layer Normalization, and Residual connections. Additionally, a small fully connected feed-forward network is added to the model, which is applied to each position separately and identically. Specifically, the model uses a Linear→ReLU→Linear MLP. The full transformation including the residual connection can be expressed as:

$$\begin{aligned} \text{FFN}(x) &= \max(0, xW_1 + b_1)W_2 + b_2 \\ x &= \text{LayerNorm}\big(x + \text{FFN}(x)\big) \end{aligned}$$

This MLP adds extra complexity to the model and allows transformations on each sequence element separately. You can imagine as this allows the model to "post-process" the new information added by the previous Multi-Head Attention, and prepare it for the next attention block. Usually, the inner dimension of the MLP is 2-8× larger than d_{model}, i.e. the dimension of the original input x. The general advantage of a wider layer instead of a narrow, multi-layer MLP is the faster, parallelizable execution.

Train and Evaluate GPT LM on GPUs

Now let's train our GPT LM!

From this homework on, we will be using a freely-available GPU, such as through Google Colab.

Why GPUs? A crucial feature of PyTorch is the support of GPUs, short for Graphics Processing Unit. A GPU can perform many thousands of small operations in parallel, making it very well suitable for performing large matrix operations in neural networks. When comparing GPUs to CPUs, we can list the following main differences:

CPUs and GPUs have both different advantages and disadvantages, which is why many computers contain both components and use them for different tasks. In case you are not familiar with GPUs, you can read up more details in this NVIDIA blog post or here.

GPUs can accelerate the training of your network up to a factor of 100 which is essential for large neural networks. PyTorch implements a lot of functionality to support GPUs (mostly those of NVIDIA due to the libraries CUDA and cuDNN).

To get a more direct comparison, let's run our model for 50 training steps with both CPU and GPU.
TODOs: Run the `cpu_gpu_comparison` in `main.py`, and fill in the running time below (will be printed out on the console):

Finally, let's run the full training and evaluation on GPU.
TODOs: Run the `gpu_full_run` function in `main.py` to train the GPT language model, paste the two generated plots here. This function will also sample from the trained GPT LM. paste the completion to the prefix after the plots.

Optional Feedback

Have feedback for this assignment? Found something confusing? We'd love to hear from you!